pfi

Python for Fixed Income

COURSE 4 · MACHINE LEARNING

Case Study: Credit Card Default

WEEK 2 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Week 2 closes: the test rows, once

18,000 training rows: fit	6,000 validation	6,000 test, once

Hands-on, 10.1 to 10.3

QuantileTransformer for heavy tails; compare_models; assert_descriptive.

Case study, 10.4 to 10.12

Three models, chosen on validation rows; the test rows opened once; people; a workbook; a description.

Allowed names

model probability of default in this file

flagged at threshold t

the model's score

Never called

a credit decision, an approval

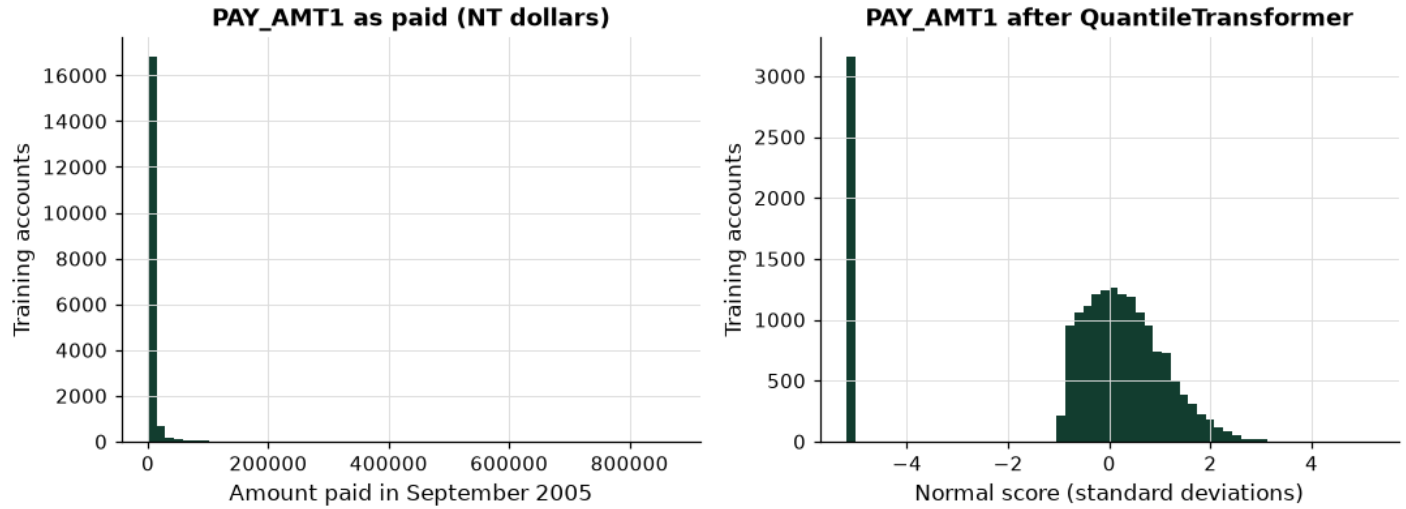
a lending policy

a risk grade for a real person

How QuantileTransformer works

The problem	A few huge amounts push a logistic regression's sum a long way; scaling keeps the gaps in proportion
Ranks	Each value is replaced by its rank among the training values
The shape	The rank is mapped onto a normal curve: the 30th percentile goes to the normal's 30th percentile
What it learns	The sorted training values only, so it is fitted inside the pipeline on the training rows
What comes out	The same order, an even spread: one large bill cannot dominate the sum

Heavy tails, and QuantileTransformer



Logistic	Scaled	Quantile
cv ROC AUC	0.7277	0.7366
ROC AUC (valid)	0.7190	0.7291
Avg precision	0.4946	0.4594

Each value becomes its quantile among the training rows, mapped to a normal curve. Order kept; gaps not. Fitted inside the pipeline.

2,100
median, NT$

873,552
largest, NT$

16.39
skew

ROC AUC up, average precision down.

compare_models: one row per model, nothing fitted

```
mlkit.compare_models(models, X, y, thresholds)  ->  one row per fitted model
```

In

Fitted models, by name. The rows to report on. A threshold per model, chosen on other rows.

Out

roc_auc and average_precision from probabilities; then accuracy, precision, recall, f1 at the model's threshold.

It never calls fit: an unfitted model raises NotFittedError. Tests: 40 passed.

assert_descriptive: a check on the words

Passes	In this file the pruned tree ranked default and non-default pairs better than the default tree.
Stopped	Accounts like these will default, so the desk should lend less to them. ['will', 'should', 'lend']
Passes	Defaults are going to rise next year.

The third passes and is not a description: it says what comes next, with no listed word.

The list is a reminder, not a proof. Read every sentence.

Case study: credit card default

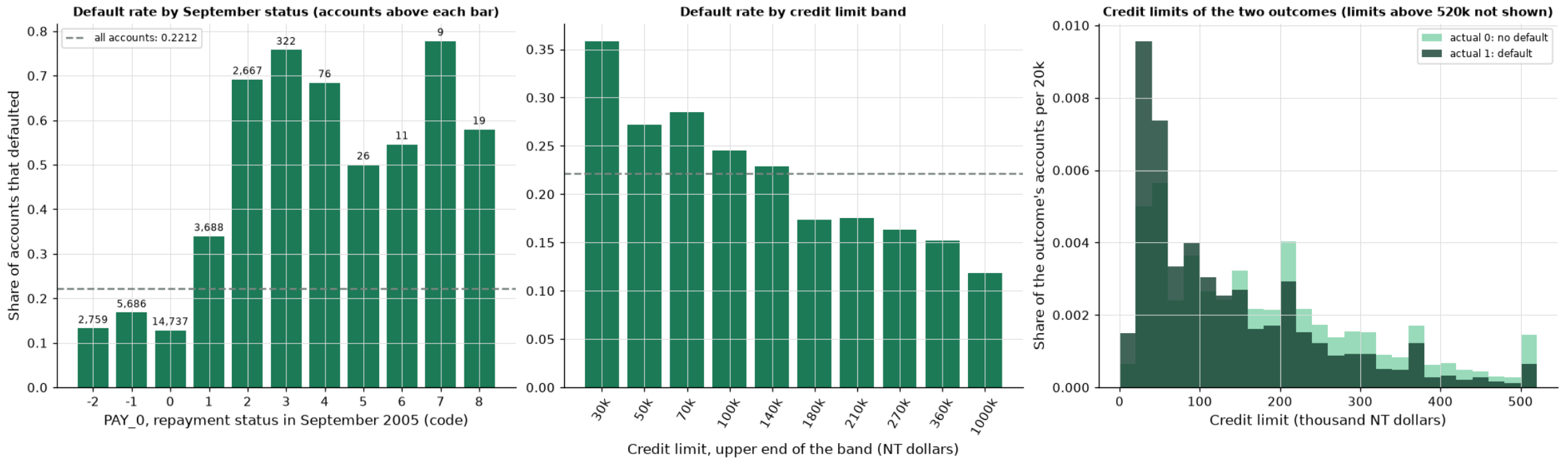
Which of three models best separates defaults, how well on unseen accounts, and where does it fail?

Step	Rows
1. Fit three models	training
2. Settings and thresholds	training folds, validation
3. Report, once	test
4. Flags by SEX label, with counts	test
5. Results workbook, opened in Excel	both
6. What it shows, where it fails	test

The file

30,000 accounts, Taiwan, 2005; 22.12% defaulted. 19 features; the four person columns never used. PAY codes -2 and 0 are not defined by the page.

The file, described



Default rate: 12.81% at code 0, 33.95% at code 1, 69.14% at code 2. By limit band: 35.85% falling to 11.87%. Description of this file, not a rule.

How GridSearchCV works

The list	A grid of candidate settings, here the ccp_alpha values
Each candidate	Fitted on four folds of the training rows, scored on the fifth, five times; the scores are averaged
The winner	The best average score; the model is then refitted with it on all the training rows
You control	The grid, the folds, and the score, here ROC AUC
What comes out	cv_results_ for every candidate, best_params_, and the refitted best_estimator_

Two trees: default and pruned

Default tree, class weights balanced

49
depth

3,233
leaves

ROC AUC 0.9997 on training rows, 0.6158 on validation. 97.62% of validation probabilities are exactly 0 or 1.

Pruned tree, ccp_alpha 0.001

6
depth

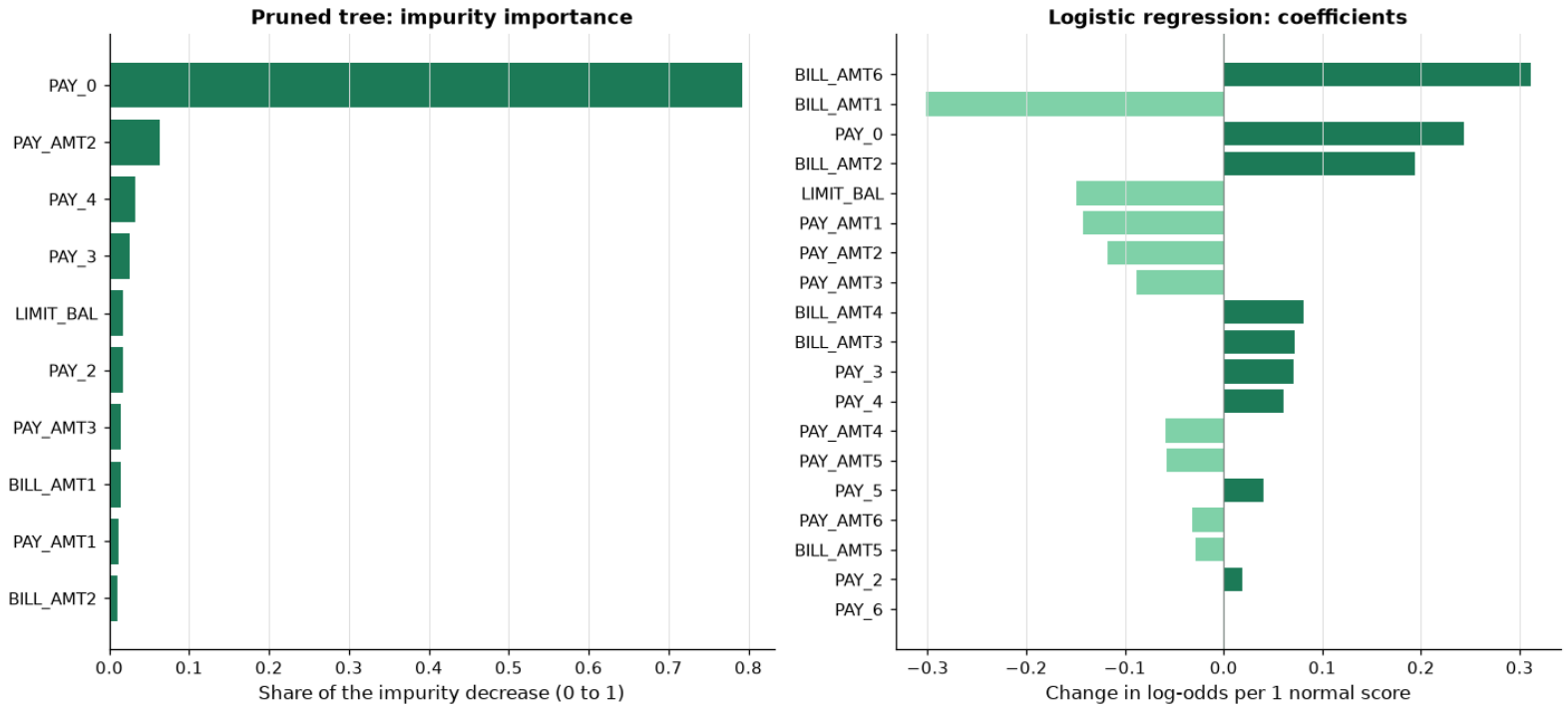
12
leaves

Chosen by GridSearchCV, five folds of the training rows, by ROC AUC.

ccp_alpha	0.0002	0.0005	0.001	0.002	0.005
cv ROC AUC	0.6838	0.7589	0.7598	0.7496	0.7369

Too little pruning keeps noise; too much cuts structure. The best alpha sits inside the grid.

What each model leans on



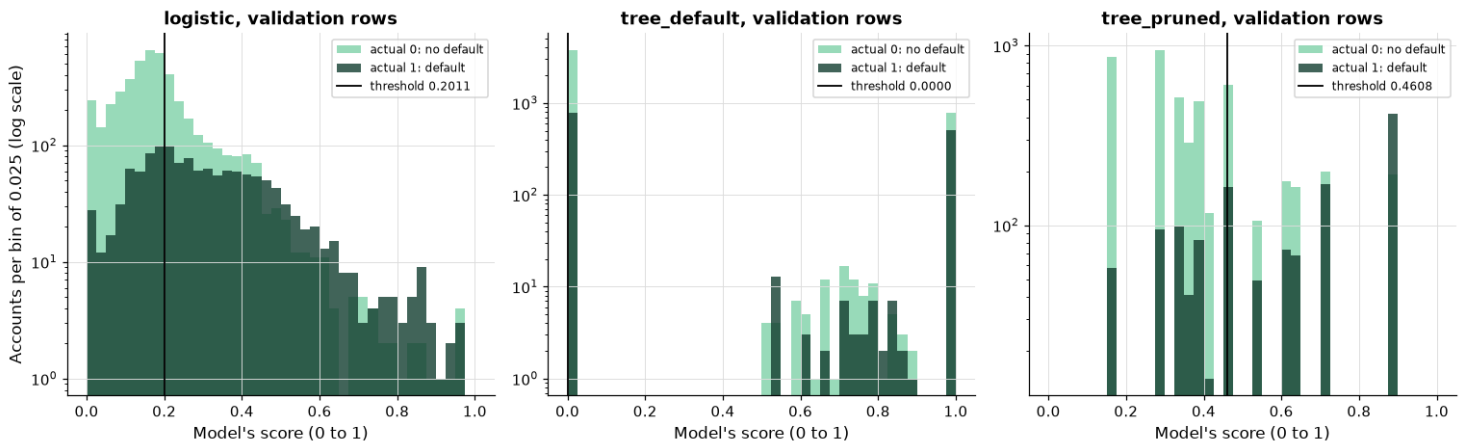
Importance has no sign; a coefficient does. Correlated bill amounts offset one another. PAY_0 matters to both.

Validation rows choose

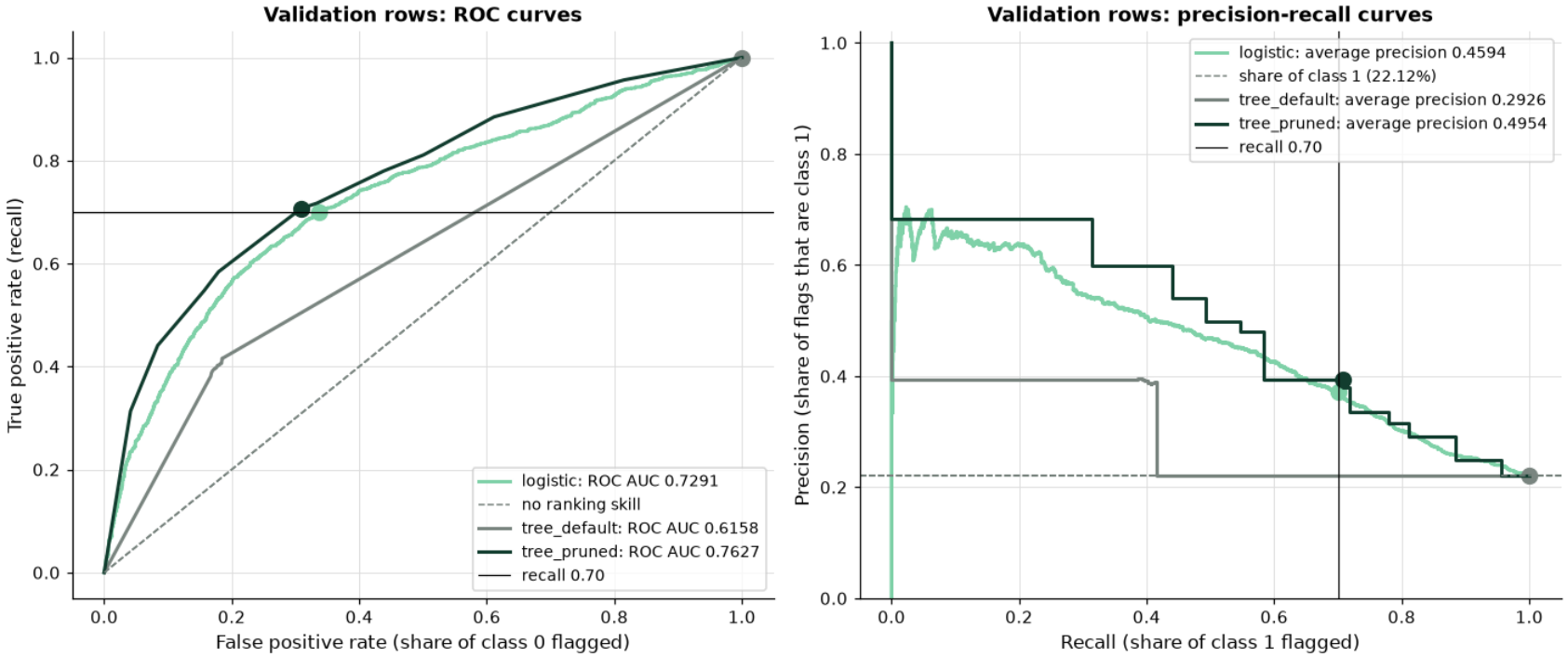
Model	ROC AUC	Avg prec	Threshold	Recall	Precision
logistic	0.7291	0.4594	0.2011	0.7001	0.3707
tree_default	0.6158	0.2926	0.0000	1.0000	0.2212
tree_pruned	0.7627	0.4954	0.4608	0.7069	0.3938

Each threshold: the highest with recall at least 0.70 on validation rows. Chosen on ROC AUC: tree_pruned.

Default tree: threshold 0.0000. Its scores are 0 or 1, so reaching 0.70 flags every account.

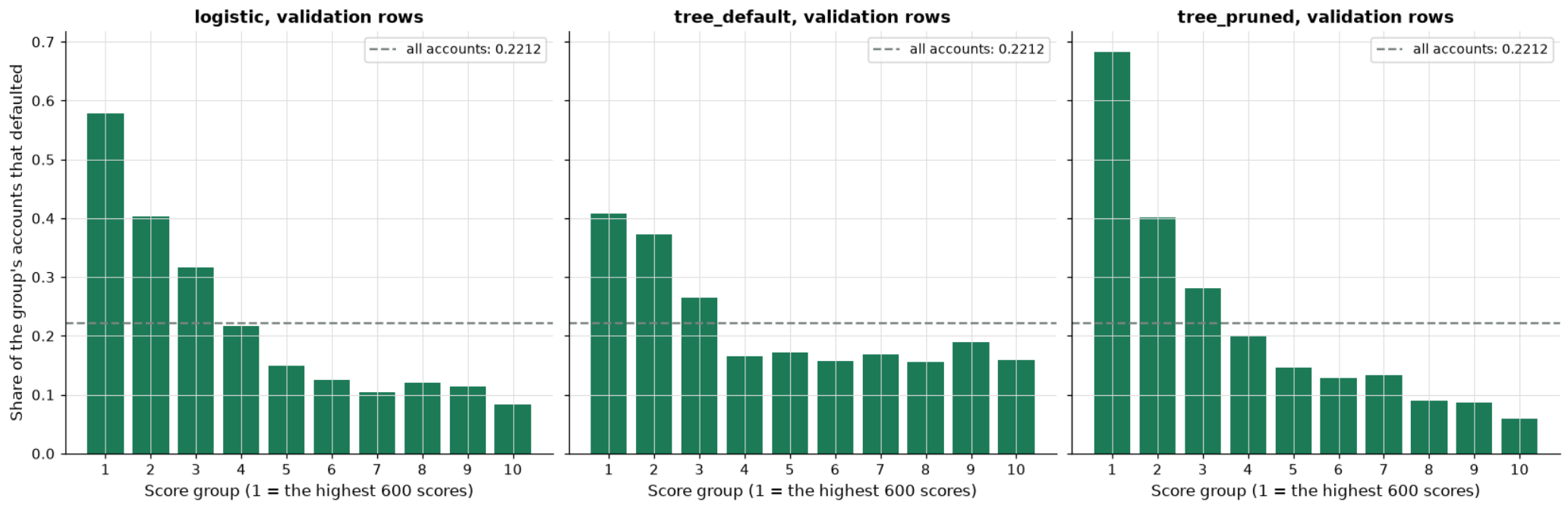


ROC and precision-recall curves, three models



Each dot is a model's threshold. At recall 0.70 the pruned tree flags fewer accounts that paid.

Does each model rank?



Top group: logistic 0.5783 (2.61 times the overall rate), pruned tree 0.6833 (3.09 times). Bottom group: 0.0833 and 0.0600. Read the default tree's ties with care.

The test rows, opened once

Model	ROC AUC	Recall	Precision	Accuracy
logistic	0.7214	0.6858	0.3571	0.6575
tree_default	0.6128	1.0000	0.2212	0.2212
tree_pruned	0.7544	0.6993	0.3799	0.6810

tree_pruned: validation to test

ROC AUC 0.7627 to 0.7544. Recall 0.7069 to 0.6993: below 0.70, and the threshold stays.

0.7788

all-zero baseline accuracy

2,443

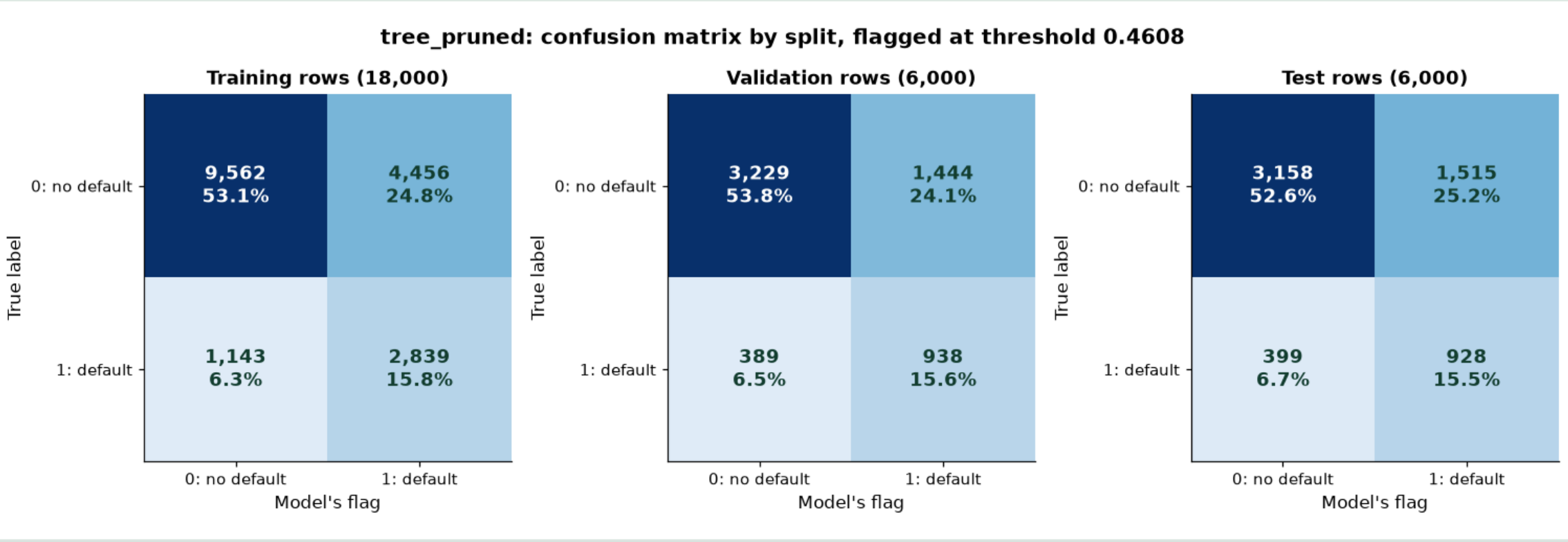
flagged of 6,000

928

defaults caught

Nothing changes after this look. Moving the threshold now would be tuning on the test set.

Training, validation, test: one row of heatmaps



Read each panel as shares: the three look alike. The test panel is the one the case study reports.

People attributes

SEX, EDUCATION, MARRIAGE, AGE: never model features in this course.

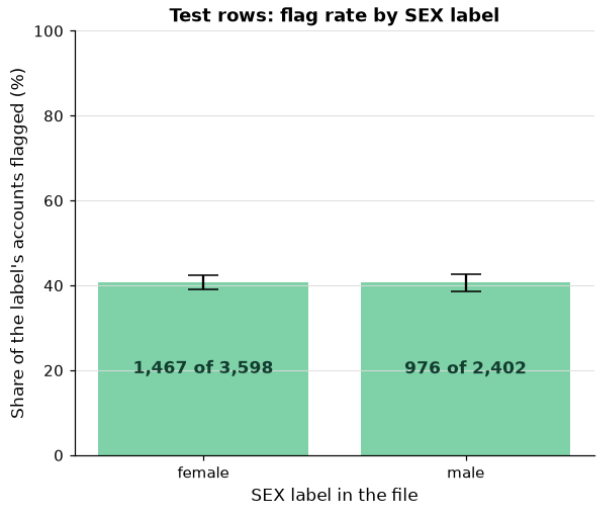
0.7366

cv ROC AUC without the four

0.7368

with them: a fact, not a reason

Label	Flag rate %	Recall %
female	40.77 +/- 1.61	70.37 +/- 3.23
male	40.63 +/- 1.96	69.34 +/- 3.82





Differences between groups in this file do not explain themselves, and they are not a basis for treating people differently.

The results workbook, in Excel

Sheet 1: comparison

One row per model and rows: validation first, then test. Column A the model; no index column.

Sheet 2: confusion

tn, fp, fn, tp on the test rows, with each model's threshold.

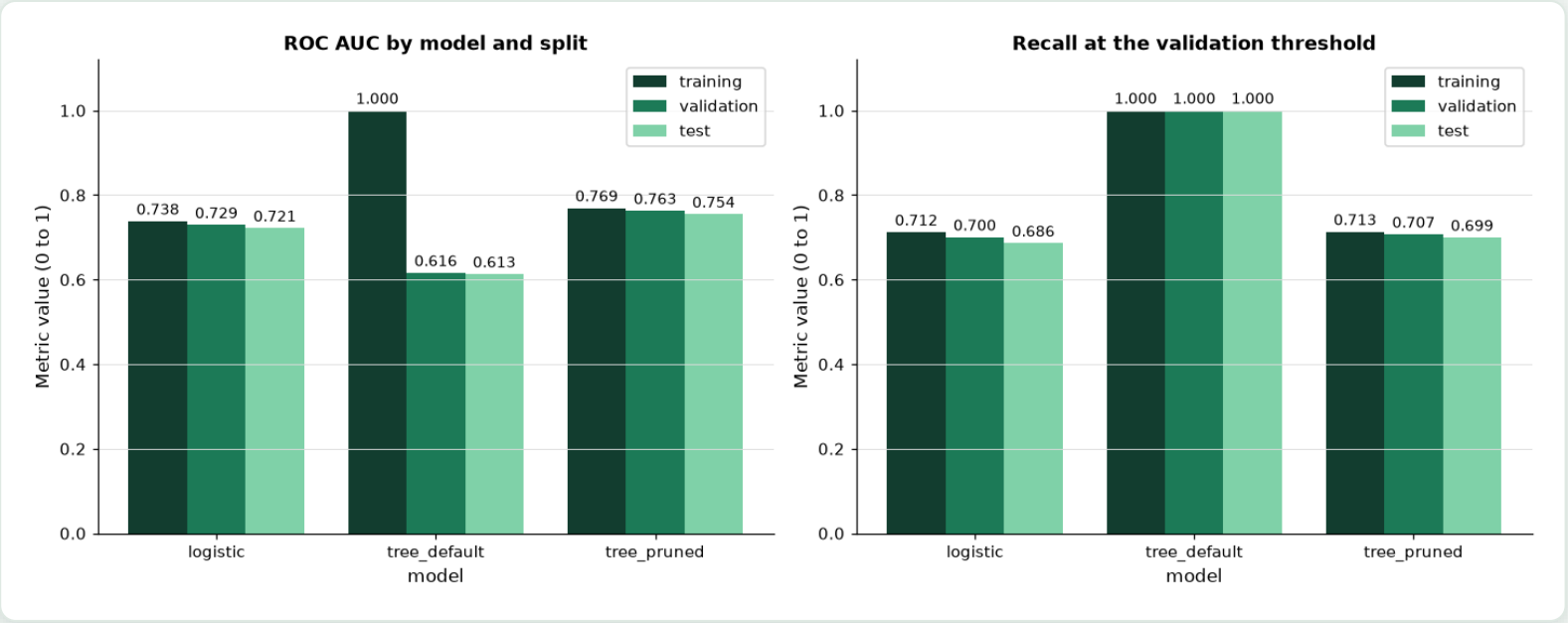
What Excel shows (General format, width 8.09):

```
tree_pruned | test | 0.754371 | 0.490291 | 0.460839 | 0.681 | 0.379861 | 0.699322 | 0.492308
```

The ROC AUC cell holds 0.754370575663462: the formula bar shows every digit. No cell showed ####. Opened with Workbooks.Open, read-only.

Tested in Excel 16.0 build 20430, 2026-10-04.

Every model, every split



ROC AUC	train	valid	test
logistic	0.7378	0.7291	0.7214
tree_default	0.9997	0.6158	0.6128
tree_pruned	0.7691	0.7627	0.7544

The default tree falls from 0.9997 on its training rows to 0.6128 on test. The pruned tree falls from 0.7691 to 0.7544.

What the model shows, and where it fails

Shows

A 12-leaf tree separates defaults better than the logistic regression and the unpruned tree, mostly through PAY_0.

Fails: On rows

379 of 399 missed defaults had a September status of 0 or below; 561 of 1,515 false flags were late in September.

Fails: On the target

Test recall 0.6993, below the 0.70 chosen on validation rows.

Fails: On the codes

The first split reads the undefined codes -2 and 0 as numbers below 1.

One place, one period, accounts chosen in a way the page does not state. No threshold here is a lending rule.

Excel side by side

Excel side by side (tested in Excel)

`=F2/(F2+D2)  (confusion sheet, H2 down)`	precision 0.3799
`=F2/(F2+E2)  (filled down)`	recall 0.6993
`=(C2+F2)/SUM(C2:F2)`	accuracy 0.6810
`=INDEX(A2:A4,MATCH(MAX(C2:C4),C2:C4,0))`	tree_pruned
`=COUNTIFS(B2:B6001,1,D2:D6001,1)/COUNTIF(B2:B6001,1)`	code 1 0.4063
`=COUNTIFS(B2:B6001,1,C2:C6001,1,D2:D6001,1)/COUNTIFS(B2:B6001,1,C2:C6001,1)`	code 1 0.6934

Excel has no logistic fit as a function and no tree: the models stay in Python, and Excel checks the results.

Save the day: close week 2

1. Code

compare_models and assert_descriptive

2. Validation

results.csv: the rows that chose

3. Test

results.csv: the one look

4. Tag

git tag week2

```
git diff --stat week1 week2
 mlkit.py      | 195 ++++++++++++++++++++++++++++++++++++++++
 results.csv   |   7 ++
 test_mlkit.py | 254 ++++++++++++++++++++++++++++++++++++++++
 3 files changed, 456 insertions(+)
```

The log's history keeps the order: choices on validation rows first, the test rows after.

The AI check

The docstring is the prompt. The code that comes back runs, and has one modelling bug.

1

Docstring

Which rows fit, which rows report, and the units: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: defaults copied before the split: 3,032 accounts on both sides, test ROC AUC 0.8518 against 0.6128.

Code that runs is not the same as code that is right.

What you can do now

- Heavy tails: QuantileTransformer, inside the pipeline
- compare_models fits nothing; thresholds come from validation
- Choice on validation, test rows once: heatmaps of all three splits
- Flags by a person label: counts, margins, no verdict
- Describe what it shows and where it fails

Week 3, Day 1 Dated data and walk-forward validation: the Treasury curve